

关系数据理论的主要内容是关系规范化理论,用来指导数据库逻辑设计,具体包括两部分:一是给出判断数据库逻辑设计“好坏程度”的准则;二是如果逻辑设计中存在“不好”的关系模式,如何将其修改为“好”的关系模式。

本章导读

在关系数据库模式设计中,我们面临两个问题,第一,什么是一个“好”的关系数据库模式,其标准是什么。如果没有判断的标准,就很难对一个具体的设计进行评价。第二,能否开发数据库设计辅助工具来减轻设计者的负担。本章6.1和6.2节介绍关系规范化理论,用于回答第一个问题;6.3和6.4节介绍函数依赖公理系统以及模式分解算法,探索自动进行模式分解的方法,用于回答第二个问题。

本章首先从数据库逻辑设计中如何构造一个好的数据库模式这一问题出发,阐明关系规范化理论研究的实际背景,然后讨论两类最重要的数据依赖——函数依赖和多值依赖。读者要牢固掌握数据依赖的基本概念,如函数依赖、部分函数依赖、完全函数依赖、传递函数依赖等,掌握1NF到4NF的概念、规范化的含义和作用;能够了解各种范式以及在插入、删除和更新数据时可能出现的问题,并直观地描述解决办法。

本章进一步讨论函数依赖的推理规则,介绍函数依赖公理系统,讲解保持函数依赖的模式等价准则及相应的模式分解算法。要掌握这些算法并在数据库模式设计中应用。6.5节是选读内容,介绍无损连接的模式分解概念及相应算法,并解释其与保持函数依赖的模式分解之间的关系。

读者学习本章后应能够根据应用语义完整地写出关系模式的数据依赖集合,并能根据数据依赖分析某一个关系模式属于第几范式,能够运用介绍的有关算法辅助设计和优化数据库模式。

6.1 问题的提出

前面已经讨论了数据库系统的一般概念,介绍了关系数据库的基本概念、关系模型的三要

素以及关系数据库的标准语言 SQL。但是还有一个很基本的问题尚未涉及：针对一个具体问题，应该如何构造一个适合它的数据库模式，即应该构造几个关系模式，每个关系模式由哪些属性组成等。如何判断这个模式是好的，也就是设计的标准如何，是关系数据库逻辑设计要解决的问题。

首先回顾一下关系模式的形式化定义。

在第2章关系数据库中已经讲过，一个关系模式可以表示为

$$R(U, D, \text{DOM}, F)$$

这里：

- ① 关系名 R 是符号化的元组语义。
- ② U 为组成该关系模式的属性名集合。
- ③ D 为 U 中的属性所来自的域。
- ④ DOM 为属性向域的映射。
- ⑤ F 为 U 上的一组数据依赖。

由于 D 、 DOM 与模式设计关系不大，因此在本章中把关系模式表示为 $R(U, F)$ 。当且仅当 U 上的一个关系 r 满足 F 时，称 r 为关系模式 $R(U, F)$ 的一个关系。

作为二维表，关系要符合一个最基本的条件是每一个分量必须是不可分的数据项。满足了这个条件的关系模式就属于第一范式(1NF)。

在模式设计中，给定一个模式 S_ϕ ，它仅由单个关系模式组成，问题是要设计一个模式 S_D ，它与 S_ϕ 等价(等价的确切含义后面再展开)，但在某些指定的方面更好一些。这里通过一个例子来说明一个不好的模式会有些什么问题，分析它们产生的原因，并从中找出设计一个好的关系模式的办法。

在举例之前，先非形式地讨论一下数据依赖的概念。

数据依赖是一个关系属性与属性之间的一种约束关系。这种约束关系是通过属性间值的相等与否体现出来的数据间的相关联系。它是现实世界属性间相互联系的抽象，是数据内在的性质和语义的体现。

人们已经提出了许多种类型的数据依赖，其中最重要的是函数依赖(functional dependency, FD)和多值依赖(multivalued dependency, MVD)。

函数依赖极为普遍地存在于现实生活中。比如描述一个学生的关系，可以有学号(Sno)、姓名(Sname)、所在学院(School)等属性。由于通常一个学号只对应一个学生，一个学生只能在一个学院学习，因而当“学号”值确定之后，学生的姓名及其他属性值也就被唯一地确定了。属性间的这种依赖关系类似于数学中的函数 $y=f(x)$ ，自变量 x 确定之后，相应的函数值 y 也就唯一地确定了。

类似的有 $\text{Sname}=f(\text{Sno})$ ， $\text{School}=f(\text{Sno})$ ，即 Sno 函数决定 Sname ， Sno 函数决定 School ，或者说 Sname 和 School 函数依赖于 Sno ，记作 $\text{Sno} \rightarrow \text{Sname}$ ， $\text{Sno} \rightarrow \text{School}$ 。

[例 6.1] 建立一个描述学校教务的数据库，该数据库涉及的对象包括学生的学号(Sno)、

所在学院(School)、所在学院院长姓名(Mname)、选修课的课程号(Cno)和成绩(Grade)。假设用一个单一的关系模式 Student 来表示,则该关系模式 Student(U, F)的属性组为

$$U = \{ \text{Sno}, \text{School}, \text{Mname}, \text{Cno}, \text{Grade} \}$$

现实世界的已知事实(语义)告诉我们:

- ① 一个学院有若干学生,但一个学生只属于一个学院。
- ② 一个学院只有一名(正职)院长。
- ③ 一个学生可以选修多门课程,每门课程有若干学生选修。
- ④ 每个学生学习每一门课程有一个成绩。

于是得到 U 上的一组函数依赖 F (如图 6.1 所示)。

$$F = \{ \text{Sno} \rightarrow \text{School}, \text{School} \rightarrow \text{Mname}, (\text{Sno}, \text{Cno}) \rightarrow \text{Grade} \}$$

如果只考虑函数依赖这一种数据依赖,可以得到一个描述学生的关系模式 Student(U, F)。

表 6.1 是某一时刻关系模式 Student 的一个实例,即数据表。

表 6.1 Student 表

Sno	School	Mname	Cno	Grade
S1	信息学院	张明	C1	95
S2	信息学院	张明	C1	90
S3	信息学院	张明	C1	88
S4	信息学院	张明	C1	70
S5	信息学院	张明	C1	78
⋮	⋮	⋮	⋮	⋮

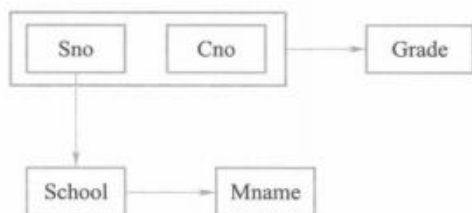


图 6.1 Student 上的一组函数依赖

但是,这个关系模式存在以下问题:

① 数据冗余(data redundancy)。比如,院长姓名重复出现,重复次数与该学院所有学生的所有课程成绩出现次数相同,如表 6.1 所示。这将浪费大量的存储空间。

② 更新异常(update anomaly)。由于数据冗余,当更新数据库中的数据时,系统要付出很大的代价来维护数据库的完整性,否则会面临数据不一致的危险。比如,某学院更换院长后,必须修改与该学院学生有关的每一个元组。

③ 插入异常(insertion anomaly)。如果一个学院刚成立,尚无学生,则无法把这个学院及其院长的信息存入数据库。

④ 删除异常(deletion anomaly)。如果某个学院的学生全部毕业了,则在删除该学院学生信息的同时,这个学院及其院长的信息也丢掉了。

鉴于存在以上种种问题,可以得出这样的结论:例 6.1 的 Student 关系模式不是一个好的模式。一个好的模式应当不会发生插入异常、删除异常和更新异常,数据冗余应尽可能

能少。

为什么会发生这些问题呢?

这是因为这个模式中的函数依赖存在某些不好的性质。这正是本章要讨论的问题。假如把这个单一的模式改造一下,分成以下三个关系模式:

```
S( { Sno, School } , { Sno → School } );
SC( { Sno, Cno, Grade } , { ( Sno, Cno ) → Grade } );
SCHOOL( { School, Mname } , { School → Mname } )
```

则这三个模式都不会发生插入异常、删除异常和更新异常的问题,数据的冗余也得到了控制。

一个模式的数据依赖会有哪些不好的性质,如何改造一个不好的模式,这就是规范化要讨论的内容。

6.2 规范化

本节首先讨论一个关系的属性间不同的依赖情况,以及如何根据属性间依赖情况判定关系是否具有某些不合适的性质。通常按属性间依赖情况区分关系的规范化程度为第一范式、第二范式、第三范式和第四范式等。然后直观地描述如何将具有不合适性质的关系转换为更合适的形式。

6.2.1 函数依赖

定义 6.1 给定关系模式 $R(U, F)$, X, Y 是 U 的子集。若对于 $R(U, F)$ 的任意一个可能的关系 r , r 中不可能存在两个元组在 X 上的属性值相等,而在 Y 上的属性值不等,则称 X 函数确定 Y 或 Y 函数依赖于 X , 记作 $X \rightarrow Y$ 。

函数依赖和别的数据依赖一样是语义范畴的概念,只能根据语义来确定一个函数依赖。例如,“姓名 $\rightarrow$ 年龄”这个函数依赖只有在该部门没有同名人的条件下成立。如果允许有同名人,则年龄就不再函数依赖于姓名了。

设计者也可以对现实世界做强制性规定,例如规定不允许同名人出现,因而使“姓名 $\rightarrow$ 年龄”函数依赖成立。这样当插入某个元组时该元组上的属性值必须满足规定的函数依赖,若发现有同名人存在,则拒绝插入该元组。

注意: 函数依赖不是指关系模式 R 的某个或某些关系满足的约束条件,而是指 R 在不同时刻的关系实例均要满足的约束条件。

例如,6.1节关系模式 $Student(U, F)$ 中有 $Sno \rightarrow School$ 成立,也就是说在任何时刻该模式的关系实例(即 $Student$ 数据表)中,不可能存在两个元组在 Sno 上的值相等,而在 $School$ 上的值不等。因此,表 6.2 的 $Student$ 表是错误的。因为表中有两个元组在 Sno 上都等于“S1”,而

在 School 上一个为“信息学院”，一个为“人工智能学院”。

表 6.2 一个错误的 Student 表

Sno	School	Mname	Cno	Grade
S1	信息学院	张明	C1	95
S1	人工智能学院	张明	C1	90
S3	信息学院	张明	C1	88
S4	信息学院	张明	C1	70
S5	信息学院	张明	C1	78
⋮	⋮	⋮	⋮	⋮

下面介绍一些术语和记号。

① $X \rightarrow Y$, 但 $Y \not\subseteq X$, 则称 $X \rightarrow Y$ 是非平凡的函数依赖(non-trivial functional dependency)。

② $X \rightarrow Y$, 但 $Y \subseteq X$, 则称 $X \rightarrow Y$ 是平凡的函数依赖(trivial functional dependency)。对于任一关系模式, 平凡的函数依赖都是必然成立的, 它不反映新的语义。若不特别声明, 总是讨论非平凡的函数依赖。

③ 若 $X \rightarrow Y$, 则 X 称为这个函数依赖的决定属性组, 也称为决定因素(determinant)。

④ 若 $X \rightarrow Y, Y \rightarrow X$, 则记作 $X \leftrightarrow Y$ 。

⑤ 若 Y 不函数依赖于 X , 则记作 $X \nrightarrow Y$ 。

定义 6.2 在 $R(U, F)$ 中, 如果 $X \rightarrow Y$, 并且对于 X 的任何一个真子集 X' , 都有 $X' \nrightarrow Y$, 则称 Y 对 X 完全函数依赖(full functional dependency), 记作 $X \xrightarrow{F} Y$ 。

若 $X \rightarrow Y$, 但 Y 不完全函数依赖于 X , 则称 Y 对 X 部分函数依赖(partial functional dependency), 记作 $X \xrightarrow{P} Y$ 。

例 6.1 中 $(Sno, Cno) \xrightarrow{F} Grade$ 是完全函数依赖, $(Sno, Cno) \xrightarrow{P} School$ 是部分函数依赖, 因为 $Sno \rightarrow School$ 成立, 而 Sno 是 (Sno, Cno) 的真子集。

定义 6.3 在 $R(U, F)$ 中, 如果 $X \rightarrow Y (Y \not\subseteq X)$, $Y \nrightarrow X$, $Y \rightarrow Z$, $Z \not\subseteq Y$, 则称 Z 对 X 传递函数依赖(transitive functional dependency)。记为 $X \xrightarrow{T} Z$ 。

这里加上条件 $Y \nrightarrow X$, 是因为如果 $Y \rightarrow X$, 则 $X \leftrightarrow Y$, 实际上是直接函数依赖而不是传递函数依赖。

例 6.1 中有 $Sno \rightarrow School$, $School \rightarrow Mname$ 成立, 所以 $Sno \xrightarrow{T} Mname$ 。

6.2.2 码^①

码是关系模式中的一个重要概念。在第2章2.1节已给出了有关码的若干描述性定义，这里用函数依赖的概念来定义码。

定义 6.4 设 K 为 $R(U, F)$ 中的属性或属性组，若 $K \xrightarrow{F} U$ ，则 K 为 R 的候选码。

注意 U 是完全函数依赖于 K ，而不是部分函数依赖于 K 。

如果 U 部分函数依赖于 K ，即 $K \xrightarrow{P} U$ ，则 K 称为超码。

候选码和超码有什么关系呢？候选码的超集（如果存在）一定是超码，候选码的任何真子集一定不是超码。

若候选码多于一个，则选定其中的一个为主码。

包含在任何一个候选码中的属性称为主属性，不包含在任何一个候选码中的属性称为非主属性或非码属性。最简单的情况，单个属性是码；最极端的情况，整个属性组是码，称为全码。

在后面的章节中主码或候选码都简称为码。

[例 6.2] 第2章的“学生”表 $Student(\underline{Sno}, Sname, Ssex, Sbirthdate, Smajor)$ 中，单个属性“ $\underline{Sno}$ ”是码，用下划线显示出来。“学生选课”表 $SC(\underline{Sno}, \underline{Cno}, Grade, Semester, Teachingclass)$ 中，属性组“ $\underline{Sno}, \underline{Cno}$ ”是码。

[例 6.3] 关系模式 $R(P, W, A)$ 中，属性 P 表示演奏者， W 表示作品， A 表示听众。假设一个演奏者可以演奏多部作品，某一部作品可被多个演奏者演奏，听众也可以欣赏不同演奏者的不同作品，这个关系模式的码为 (P, W, A) ，即全码。

定义 6.5 关系模式 R 中属性或属性组 X 并非 R 的码，但 X 是另一个关系模式的码，则称 X 是 R 的外码。

如第2章的“学生选课”表 $SC(\underline{Sno}, \underline{Cno}, Grade, Semester, Teachingclass)$ 中， Sno 不是码，但 Sno 是关系模式 $Student(\underline{Sno}, Sname, Ssex, Sbirthdate, Smajor)$ 的码，则 Sno 是关系模式 SC 的外码。主码与外码提供了一个表示关系间联系的手段。

6.2.3 范式

关系数据库中的关系是要满足一定要求的，满足不同程度要求的关系为不同范式。满足最低要求的关系叫第一范式，简称 1NF；在第一范式中满足进一步要求的关系为第二范式，其余以此类推。

有关范式理论的研究主要是 Edgar F. Codd 做的工作。1971—1972 年 Codd 系统地提出了 1NF、2NF、3NF 的概念，讨论了规范化的问题。1974 年，Codd 和 Boyce 共同提出了一个新范

^① 在一些教材和文章中也将码称为“键”或“键码”。

式, 即 BCNF。1976 年 Fagin 提出了 4NF。

所谓“第几范式”原本是表示关系的某一种级别, 所以常称某一关系模式 R 为第几范式。现在则把范式这个概念理解成符合某一种级别的关系模式的集合, 即 R 为第几范式就可以写成 $R \in xNF$ 。

对于各种范式之间的关系有

$$4NF \subset BCNF \subset 3NF \subset 2NF \subset 1NF$$

成立, 如图 6.2 所示。

一个低一级范式的关系模式通过模式分解 (schema decomposition) 可以转换为若干个更高级别范式的关系模式的集合, 这种过程就叫规范化。

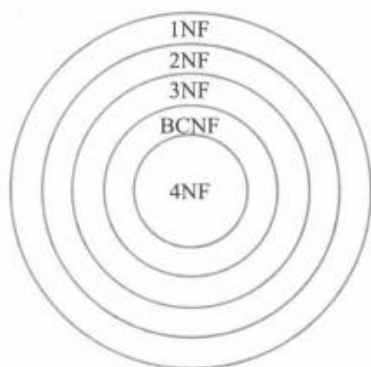


图 6.2 各种范式之间的关系

6.2.4 2NF

如果关系模式 R 的每一个分量是一个不可分的数据项, R 是满足最低要求的关系模式, 则 $R \in 1NF$ 。

定义 6.6 若 $R \in 1NF$, 且每一个非主属性完全函数依赖于任何一个候选码, 则 $R \in 2NF$ 。

下面举一个不是 2NF 的例子。

[例 6.4] 有关系模式 $S-L-C(Sno, School, Sloc, Cno, Grade)$, 其中 $Sloc$ 为学生的住所, 并且每个学院的学生住在同一个宿舍楼。 $S-L-C$ 的码为 (Sno, Cno) 。则函数依赖有

$$(Sno, Cno) \xrightarrow{F} Grade$$

$$Sno \rightarrow School, (Sno, Cno) \xrightarrow{P} School$$

$$Sno \rightarrow Sloc, (Sno, Cno) \xrightarrow{P} Sloc, \\ School \rightarrow Sloc \text{ (每个学院的学生住同一个宿舍楼)}$$

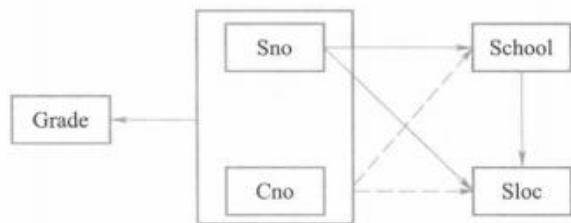


图 6.3 函数依赖示例

函数依赖关系如图 6.3 所示。

图中用虚线表示部分函数依赖。另外, $School$ 函数确定 $Sloc$, 这一点在讨论第二范式时暂不考虑。可以看到, 非主属性 $School$ 、 $Sloc$ 并不完全函数依赖于码。因此 $S-L-C(Sno, School, Sloc, Cno, Grade)$ 不符合 2NF 定义, 即 $S-L-C \notin 2NF$ 。

一个关系模式 R 不属于 2NF, 就会产生以下几个问题:

① 插入异常。假若要插入一个学生 $Sno=S7$, $School=INF$, $Sloc=BLD2$, 但该生还未选课, 即这个学生无 Cno , 这样的元组就插不进 $S-L-C$ 中。因为插入元组时必须给定码值, 而这时码值的一部分为空, 因而学生的固有信息无法插入。

② 删除异常。假定某个学生只选一门课，如 S4 就选了一门课 C3，现在 C3 这门课他也不选了，那么 C3 这个数据项就要删除。而 C3 是主属性，删除了 C3，整个元组就必须一起删除，使得 S4 的其他信息也被删除了，从而造成删除异常，即不应删除的信息也删除了。

③ 修改复杂。某个学生从人工智能学院转到信息学院，这本来只需修改此学生元组中的 School 分量即可，但因为关系模式 S-L-C 中还含有学生的住所 Sloc 属性，学生转学院将同时改变住所，因而还必须修改元组中的 Sloc 分量。另外，如果这个学生选修了 k 门课，School、Sloc 重复存储了 k 次，不仅存储冗余度大，而且必须无遗漏地修改 k 个元组中的全部 School、Sloc 信息，造成修改的复杂化。

分析上面的例子可以发现问题在于有两类非主属性，一类如 Grade，它对码是完全函数依赖；另一类如 School、Sloc，它们对码不是完全函数依赖。解决的办法是用投影分解把关系模式 S-L-C 分解为两个关系模式：SC(Sno, Cno, Grade) 和 S-L(Sno, School, Sloc)。

关系模式 SC 与 S-L 中属性间的函数依赖可以用图 6.4 和图 6.5 表示如下。

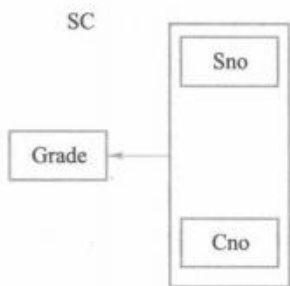


图 6.4 SC 中的函数依赖

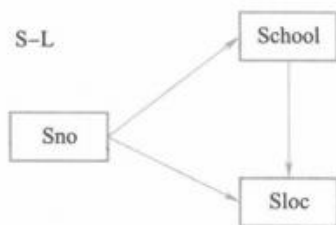


图 6.5 S-L 中的函数依赖

关系模式 SC 的码为 (Sno, Cno)，关系模式 S-L 的码为 Sno，这样就使得非主属性对码都是完全函数依赖了。

6.2.5 3NF

定义 6.7 设关系模式 $R(U, F) \in 1NF$ ，若 R 中不存在这样的码 X ，属性组 Y 及非主属性 $Z (Y \not\subseteq Z)$ ，使得 $X \rightarrow Y$ ， $Y \rightarrow Z$ 成立， $Y \not\rightarrow X$ ，则称 $R(U, F) \in 3NF$ 。

由定义 6.7 可以证明，若 $R \in 3NF$ ，则每一个非主属性既不传递依赖于码，也不部分依赖于码。也就是说，可以证明如果 R 属于 3NF，则必有 R 属于 2NF。

在图 6.4 中关系模式 SC 没有传递依赖，而图 6.5 中关系模式 S-L 存在非主属性对码的传递依赖。在 S-L 中，由 $Sno \rightarrow School$ ($School \not\rightarrow Sno$)， $School \rightarrow Sloc$ ，可得 $Sno \xrightarrow{T} Sloc$ 。

因此 $SC \in 3NF$ ，而 $S-L \notin 3NF$ 。

一个关系模式 R 若不是 3NF, 就会产生与 6.2.4 节中 2NF 相类似的问题。读者可以类比 2NF 的反例加以说明。

解决的办法同样是将 $S-L$ 分解为 $S-SS(Sno, School)$ 和 $S-SL(School, Sloc)$ 。分解后的关系模式 $S-SS$ 与 $S-SL$ 中不再存在传递依赖。

6.2.6 BCNF

BCNF(Boyce Codd Normal Form)是由 Boyce 与 Codd 提出的, 比上述 3NF 又进了一步, 通常认为 BCNF 是修正的第三范式, 有时也称为扩充的第三范式。

定义 6.8 关系模式 $R(U, F) \in 1NF$, 若 $X \rightarrow Y$ 且 $Y \not\subseteq X$ 时 X 必含有码, 则 $R(U, F) \in BCNF$ 。也就是说, 关系模式 $R(U, F)$ 中, 若每一个决定因素都包含码, 则 $R(U, F) \in BCNF$ 。

由 BCNF 的定义可以得到结论, 一个满足 BCNF 的关系模式有:

- ① 所有非主属性对每一个码都是完全函数依赖。
- ② 所有主属性对每一个不包含它的码也是完全函数依赖。
- ③ 没有任何属性完全函数依赖于非码的任何一组属性。

由于 $R \in BCNF$, 按定义排除了任何属性对码的传递依赖与部分依赖, 所以 $R \in 3NF$ 。严格的证明留给读者完成。但是若 $R \in 3NF$, R 未必属于 BCNF。

下面用几个例子说明属于 3NF 的关系模式有的属于 BCNF, 但有的不属于 BCNF。

[例 6.5] 考察关系模式 $Course(\underline{Cno}, Cname, Ccredit, Cpro)$, 它只有一个码 Cno , 这里没有任何属性对 Cno 部分依赖或传递依赖, 所以 $Course \in 3NF$ 。同时 $Course$ 中 Cno 是唯一的决定因素, 所以 $Course \in BCNF$ 。

对于关系模式 $SC(\underline{Sno}, \underline{Cno}, Grade, Semester, Teachingclass)$ 可做同样分析。

[例 6.6] 关系模式 $Student(\underline{Sno}, Sname, Ssex, Sbirthdate, Smajor)$ 中, 假定 $Sname$ 也具有唯一性, 那么 $Student$ 就有两个候选码, 这两个码都由单个属性组成, 彼此不相交。其他属性不存在对码的传递依赖与部分依赖, 所以 $Student \in 3NF$ 。同时 $Student$ 中除 Sno 、 $Sname$ 外没有其他决定因素, 所以 $Student \in BCNF$ 。

以下再举几个例子。

[例 6.7] 关系模式 $SJP(S, J, P)$ 中, S 是学生, J 表示课程, P 表示名次。每一个学生选修每门课程的成绩有一定的名次, 每门课程中每一名次只有一个学生(即没有并列名次)。由语义可得到下面的函数依赖:

$$(S, J) \rightarrow P; (J, P) \rightarrow S$$

所以 (S, J) 与 (J, P) 都可以作为候选码。这两个码各由两个属性组成, 而且它们是相交的。这个关系模式中显然没有属性对码传递依赖或部分依赖, 所以 $SJP \in 3NF$, 而且除 (S, J) 与 (J, P) 以外没有其他决定因素, 所以 $SJP \in BCNF$ 。

[例 6.8] 关系模式 $STJ(S, T, J)$ 中, S 表示学生, T 表示教师, J 表示课程。每一教师只教一门课, 每门课有若干教师, 某一学生选定某门课就对应一个固定的教师。由语义可得到如下的函数依赖:

$$(S, J) \rightarrow T, (S, T) \rightarrow J, T \rightarrow J$$

函数依赖关系可以用图 6.6 表示, 这里 (S, J) 、 (S, T) 都是候选码。

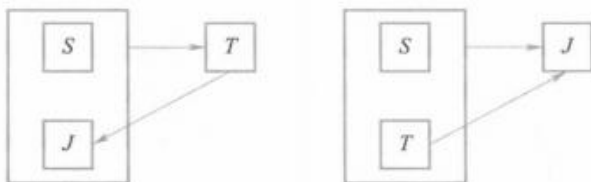


图 6.6 STJ 中的函数依赖

STJ 是 3NF, 因为没有任何非主属性对码传递依赖或部分依赖, 但 STJ 不是 BCNF 关系, 因为 T 是决定因素, 而 T 不包含码。

对于不是 BCNF 的关系模式, 仍然存在不合适的地方。读者可自己举例指出 STJ 的不合适之处。非 BCNF 的关系模式也可以通过分解成为 BCNF。例如 STJ 可分解为 $ST(S, T)$ 与 $TJ(T, J)$, 它们都是 BCNF。

3NF 和 BCNF 是在函数依赖的条件下对模式分解所能达到的分离程度的测度。一个模式中的关系模式如果都属于 BCNF, 那么在函数依赖范畴内它已实现了彻底分离, 已消除了插入和删除的异常。3NF 的“不彻底”性表现在可能存在主属性对码的部分依赖和传递依赖。

6.2.7 多值依赖

以上是在函数依赖的范畴内讨论问题。属于 BCNF 的关系模式是否就很完美了呢? 下面讲解多值依赖。首先来看一个例子。

[例 6.9] 学校中如果某一门课程由多个教师讲授, 他们使用一套相同的参考书。每个教师可以讲授多门课程, 每种参考书可以供多门课程使用。可以用一个非规范化的关系来描述课程 C 、教师 T 和参考书 B 之间的关系, 如表 6.3 所示。

把这张表变成一张规范化的二维表 Teaching, 如表 6.4 所示。

关系模型 Teaching($\underline{C}, T, B$) 的码是 ($\underline{C}, T, B$), 即全码, 因而 Teaching $\in$ BCNF。但是当某一课程(如物理)增加一名讲课教师(如周英)时, 必须插入多个(这里是三个)元组: (物理, 周英, 普通物理学), (物理, 周英, 光学原理), (物理, 周英, 物理习题集)。

同样, 某一门课程(如数学)要去掉一本参考书(如微分方程), 则必须删除多个(这里是两个)元组: (数学, 李勇, 微分方程), (数学, 张平, 微分方程)。

因而对数据的增、删、改很不方便, 数据的冗余也十分明显。仔细考察这类关系模式, 发现它具有有一种称之为多值依赖的数据依赖。

表 6.3 非规范化关系示例

课程 C	教师 T	参考书 B
物理	{李勇 王军}	{普通物理学 光学原理 物理习题集}
数学	{李勇 张平}	{数学分析 微分方程 高等代数}
计算数学	{张平 周峰}	{数学分析}
⋮	⋮	⋮

表 6.4 规范化的二维表 Teaching

课程 C	教师 T	参考书 B
物理	李勇	普通物理学
物理	李勇	光学原理
物理	李勇	物理习题集
物理	王军	普通物理学
物理	王军	光学原理
物理	王军	物理习题集
数学	李勇	数学分析
数学	李勇	微分方程
数学	李勇	高等代数
数学	张平	数学分析
数学	张平	微分方程
数学	张平	高等代数
⋮	⋮	⋮

定义 6.9 给定关系模式 $R(U, F)$, X, Y, Z 是 U 的子集, 并且 $Z = U - X - Y$ 。关系模式中多值依赖 $X \twoheadrightarrow Y$ 成立, 当且仅当对 $R(U, F)$ 的任一关系 r 给定的一对 (x, z) 值, 有一组 Y 的值, 这组值仅仅决定于 x 值而与 z 值无关。

例如, 在表 6.4 的关系模式 Teaching 中, 对于一个 (物理, 光学原理) 有一组 T 值 {李勇, 王军}, 这组值仅仅决定于课程 C 上的值 (物理)。也就是说, 对于另一个 (物理, 普通物理学), 它对应的一组 T 值仍是 {李勇, 王军}, 尽管这时参考书 B 的值已经改变了。因此 T 多值依赖于 C , 即 $C \twoheadrightarrow T$ 。

对于多值依赖的另一个等价的形式化定义是: 在 $R(U, F)$ 的任一关系 r 中, 如果存在元组 t, s , 使得 $t[X] = s[X]$, 那么就必然存在元组 $w, v \in r$ (w, v 可以与 s, t 相同), 使得 $w[X] = v[X] = t[X]$, 而 $w[Y] = t[Y]$, $w[Z] = s[Z]$, $v[Y] = s[Y]$, $v[Z] = t[Z]$ (即交换 s, t 元组的 Y 值所得的两个新元组必在 r 中), 则 Y 多值依赖于 X , 记为 $X \twoheadrightarrow Y$ 。这里, X, Y 是 U 的子集, $Z = U - X - Y$ 。

若 $X \twoheadrightarrow Y$, 而 $Z = \phi$, 即 Z 为空, 则称 $X \twoheadrightarrow Y$ 为平凡的多值依赖。即对于 $R(X, Y)$, 如果有 $X \twoheadrightarrow Y$ 成立, 则 $X \twoheadrightarrow Y$ 为平凡的多值依赖。

下面再举一个具有多值依赖的关系模式的例子。

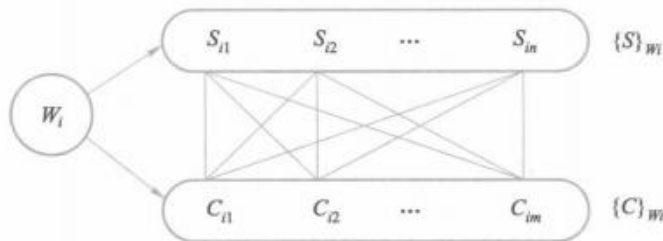
[例 6.10] 关系模式 WSC(W, S, C) 中, W 表示仓库, S 表示保管员, C 表示商品。假设每个仓库有若干个保管员和若干种商品, 每个保管员保管所在仓库的所有商品, 每种商品被所有保管员保管。列出关系如表 6.5 所示。

按照语义, 对于 W 的每一个值 W_i , S 有一个完整的集合与之对应而不问 C 的取何值, 所以 $W \twoheadrightarrow S$ 。

如果用图 6.7 来表示这种对应, 则对应 W 的某一个值 W_i 的全部 S 值记作 $\{S\}_{W_i}$ (表示此仓库工作的全部保管员), 全部 C 值记作 $\{C\}_{W_i}$ (表示在此仓库中存放的所有商品)。应当有 $\{S\}_{W_i}$ 中的每一个值和 $\{C\}_{W_i}$ 中的每一个 C 值对应。于是 $\{S\}_{W_i}$ 与 $\{C\}_{W_i}$ 之间正好形成一个完全二分图, 因而 $W \twoheadrightarrow S$ 。

表 6.5 WSC 表

W	S	C
W_1	S_1	C_1
W_1	S_1	C_2
W_1	S_1	C_3
W_1	S_2	C_1
W_1	S_2	C_2
W_1	S_2	C_3
W_2	S_3	C_4
W_2	S_3	C_5
W_2	S_4	C_4
W_2	S_4	C_5

图 6.7 $W \twoheadrightarrow S$ 且 $W \twoheadrightarrow C$

直观上, 一个关系模式中的两个属性如果彼此之间没有任何联系, 则它们是相互独立的。但如果它们都和一个共同的属性有关, 则将这两个无关的属性放在一个关系中时会造成存储的冗余。在上面的这个例子中, 保管员 S 和商品 C 是相互独立的, 但是它们都和仓库 W 有联系, W 确定了, 则有一组保管员和一批商品与之对应。但是每一个保管员需要管理该仓库中的所有商品, 换言之, 保管员和商品之间在确定的仓库下是完全二分图的联系, 表达这样的联系会浪费存储空间, 没有实际的意义。在这样的情况下, 将其分离存储很有必要。也就是, 把 W 和 S 组成一个关系, W 和 C 组成另一个关系。

由于 C 与 S 的完全对称性, 必然有 $W \twoheadrightarrow C$ 成立。

多值依赖具有以下性质:

① 多值依赖具有对称性。即若 $X \twoheadrightarrow Y$, 则 $X \twoheadrightarrow Z$, 其中 $Z = U - X - Y$ 。

从例 6.10 容易看出, 因为每个保管员保管所有商品, 同时每种商品被所有保管员保管, 显然若 $W \twoheadrightarrow S$, 必然有 $W \twoheadrightarrow C$ 。

② 多值依赖具有传递性。即如果 $XYZ = U$, $X \twoheadrightarrow Y$, $Y \twoheadrightarrow Z$, 则 $X \twoheadrightarrow Z - Y$ 。

③ 函数依赖可以看作是多值依赖的特殊情况, 即若 $X \rightarrow Y$, 则 $X \twoheadrightarrow Y$ 。这是因为当 $X \rightarrow Y$ 时, 对 X 的每一个值 x , Y 有一个确定的值 y 与之对应, 故 $X \twoheadrightarrow Y$ 。

④ 若 $X \twoheadrightarrow Y$, $X \twoheadrightarrow Z$, 则 $X \twoheadrightarrow YZ$ 。

⑤ 若 $X \twoheadrightarrow Y$, $X \twoheadrightarrow Z$, 则 $X \twoheadrightarrow Y \cap Z$ 。

⑥ 若 $X \twoheadrightarrow Y$, $X \twoheadrightarrow Z$, 则 $X \twoheadrightarrow Y-Z$, $X \twoheadrightarrow Z-Y$ 。

多值依赖与函数依赖相比, 具有下面两个基本的区别:

① 多值依赖的有效性与属性组的范围有关。若 $X \twoheadrightarrow Y$ 在 U 上成立, 则在 $W (XY \subseteq W \subseteq U)$ 上一定成立; 反之则不然, 即 $X \twoheadrightarrow Y$ 在 $W (W \subset U)$ 上成立, 在 U 上并不一定成立。这是因为多值依赖的定义中不仅涉及 U 的子集 X 和 Y , 而且还涉及 U 中其余的子集 Z 。

一般地, 在 $R(U, F)$ 上若有 $X \twoheadrightarrow Y$ 在 $W (W \subset U)$ 上成立, 则称 $X \twoheadrightarrow Y$ 为 $R(U, F)$ 的嵌入型多值依赖。

但是在关系模式 $R(U, F)$ 中, 函数依赖 $X \rightarrow Y$ 的有效性仅决定于 X 、 Y 的值。只要在 $R(U, F)$ 的任何一个关系 r 中, 元组在 X 和 Y 上的值满足定义 6.1, 则函数依赖 $X \rightarrow Y$ 在任何属性组 $W (XY \subseteq W \subseteq U)$ 上成立。

② 若函数依赖 $X \rightarrow Y$ 在 $R(U, F)$ 上成立, 则对于任何 $Y' \subset Y$ 均有 $X \rightarrow Y'$ 成立。而多值依赖 $X \twoheadrightarrow Y$ 若在 $R(U, F)$ 上成立, 却不能断言对于任何 $Y' \subset Y$ 有 $X \twoheadrightarrow Y'$ 成立。

例如, 有关系 $R(A, B, C, D)$, $A \twoheadrightarrow BC$, 当然也有 $A \twoheadrightarrow D$ 成立。有 R 的一个关系实例, 在此实例上 $A \twoheadrightarrow B$ 是不成立的, 如表 6.6 所示。

表 6.6 R 的一个实例

A	B	C	D
a_1	b_1	c_1	d_1
a_1	b_1	c_1	d_2
a_1	b_2	c_2	d_1
a_1	b_2	c_2	d_2

6.2.8 4NF

定义 6.10 关系模式 $R(U, F) \in 1NF$, 如果对于 R 的每个非平凡多值依赖 $X \twoheadrightarrow Y (Y \not\subseteq X)$, X 都含有码, 则称 $R(U, F) \in 4NF$ 。

4NF 就是限制关系模式的属性之间不允许有非平凡且非函数依赖的多值依赖。因为根据定义, 对于每一个非平凡的多值依赖 $X \twoheadrightarrow Y$, X 都含有候选码, 于是就有 $X \rightarrow Y$, 所以 4NF 所允许的非平凡的多值依赖实际上是函数依赖。

显然, 如果一个关系模式是 4NF, 则必为 BCNF。

在前面讨论的关系模式 WSC 中, $W \twoheadrightarrow S$, $W \twoheadrightarrow C$, 它们都是非平凡的多值依赖。而 W 不是码, 关系模式 WSC 的码是 $(\underline{W}, S, C)$, 即全码。因此 $WSC \notin 4NF$ 。

一个关系模式如果已达到了 BCNF 但不是 4NF, 这样的关系模式仍然具有不好的性质。以 WSC 为例, $WSC \notin 4NF$, 但是 $WSC \in BCNF$ 。对于 WSC 的某个关系, 若某一仓库 W_i 有 n 个保管员, 存放 m 件物品, 则关系中分量为 W_i 的元组数目一定有 $m \times n$ 个。每个保管员重复存储 m 次, 每种物品重复存储 n 次, 数据的冗余度太大, 因此还应该继续规范化使关系模式 WSC 达到 4NF。

可以用投影分解的方法消去非平凡且非函数依赖的多值依赖。例如,可以把WSC分解为 $WS(W,S)$, $WC(W,C)$ 。在WS中虽然有 $W \twoheadrightarrow S$,但这是平凡的多值依赖。WS中已不存在非平凡的非函数依赖的多值依赖,所以 $WS \in 4NF$,同理 $WC \in 4NF$ 。

函数依赖和多值依赖是两种最重要的数据依赖。如果只考虑函数依赖,则属于BCNF的关系模式规范化程度已经是最高了;如果考虑多值依赖,则属于4NF的关系模式规范化程度是最高的。事实上,数据依赖中除函数依赖和多值依赖之外,还有其他数据依赖,例如有一种连接依赖。函数依赖是多值依赖的一种特殊情况,而多值依赖实际上又是连接依赖的一种特殊情况。但连接依赖不像函数依赖和多值依赖一样可由语义直接导出,而是在关系的连接运算时才反映出来。存在连接依赖的关系模式仍可能遇到数据冗余及插入、修改、删除异常等问题。如果消除了属于4NF的关系模式中存在的连接依赖,则可以进一步达到5NF的关系模式。这里不再讨论连接依赖和5NF,有兴趣的读者可以参阅有关书籍。

规范化小结

在关系数据库中,对关系模式的基本要求是满足第一范式,这样的关系模式就是合法的、允许的。但是,人们发现有些关系模式存在插入、删除异常,以及修改复杂、数据冗余等问题,需要寻求解决这些问题的方法,这就是规范化的目的。

规范化的基本思想是逐步消除数据依赖中不合适的部分,使模式中的各关系模式达到某种程度的“分离”,即“一事一地”的模式设计原则。让一个关系描述一个概念、一个实体或者实体间的一种联系。若多于一个概念就把它“分离”出去。因此,所谓规范化实质上是概念的单一化。

人们认识这个原则是经历了一个过程的。从认识非主属性的部分函数依赖的危害开始,2NF、3NF、BCNF、4NF的相继提出是这个认识过程逐步深化的标志,图6.8可以概括这个过程。

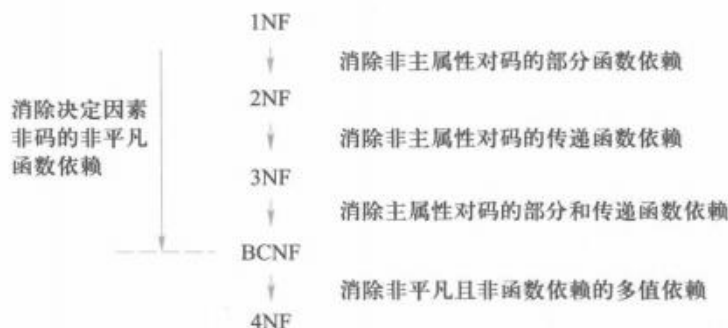


图 6.8 规范化过程

关系模式的规范化过程是通过对关系模式的分解来实现的,即把低一级的关系模式分解为若干个更高级别的关系模式。因此,关系规范化程度可以自然地用于设计“好”的关系模式标准,规范化程度越高,可能的数据异常就越少。

6.3 数据依赖的公理系统

在明确了设计标准以后,紧接着一个问题就是,是否可以找到一个算法,通过分解模式来获得达到某个规范化程度(比如3NF)的模式集合呢?如果存在这样的算法,就可以开发出关系模式的自动化设计工具,以减轻设计人员的负担。

数据依赖的公理系统是模式分解算法的理论基础。下面首先讨论函数依赖的一个有效而完备的公理系统——Armstrong 公理系统。

定义 6.11 给定关系模式 $R(U, F)$, 其任何一个关系 r , 若函数依赖 $X \rightarrow Y$ 都成立(即 r 中任意两元组 t, s , 若 $t[X] = s[X]$, 则 $t[Y] = s[Y]$), 则称 F 逻辑蕴涵 $X \rightarrow Y$ 。

为了从一组函数依赖求得蕴涵的函数依赖,例如已知函数依赖集 F , 要问 $X \rightarrow Y$ 是否为 F 所蕴涵,就需要一套推理规则,这组推理规则是1974年首先由 Armstrong 提出来的。

Armstrong 公理系统(Armstrong's axiom) 设 U 为属性组全集, F 是 U 上的一组函数依赖,于是有关系模式 $R(U, F)$, 对 $R(U, F)$ 来说有以下的推理规则。

A1 自反律(reflexivity rule): 若 $Y \subseteq X \subseteq U$, 则 $X \rightarrow Y$ 为 F 所蕴涵。

A2 增广律(augmentation rule): 若 $X \rightarrow Y$ 为 F 所蕴涵, 且 $Z \subseteq U$, 则 $XZ \rightarrow YZ$ ^① 为 F 所蕴涵。

A3 传递律(transitivity rule): 若 $X \rightarrow Y$ 及 $Y \rightarrow Z$ 为 F 所蕴涵, 则 $X \rightarrow Z$ 为 F 所蕴涵。

注意: 由自反律所得到的函数依赖均是平凡的函数依赖, 自反律的使用并不依赖于 F 。

定理 6.1 Armstrong 推理规则是正确的。

下面从定义出发证明推理规则的正确性。

证

(1) 设 $Y \subseteq X \subseteq U$

对 $R(U, F)$ 的任一关系 r 中的任意两个元组 t, s :

若 $t[X] = s[X]$, 由于 $Y \subseteq X$, 有 $t[Y] = s[Y]$,

所以 $X \rightarrow Y$ 成立, 自反律得证^②。

(2) 设 $X \rightarrow Y$ 为 F 所蕴涵, 且 $Z \subseteq U$

设 $R(U, F)$ 的任一关系 r 中的任意两个元组 t, s :

若 $t[XZ] = s[XZ]$, 则有 $t[X] = s[X]$ 和 $t[Z] = s[Z]$;

由 $X \rightarrow Y$, 于是有 $t[Y] = s[Y]$, 所以 $t[YZ] = s[YZ]$, $XZ \rightarrow YZ$ 为 F 所蕴涵, 增广律得证。

① 为了简单起见, 用 XZ 代表 $X \cup Z$, YZ 代表 $Y \cup Z$ 。

② $t[X]$ 表示元组 t 在属性(组) X 上的分量, 等价于 $t.X$ 。

(3) 设 $X \rightarrow Y$ 及 $Y \rightarrow Z$ 为 F 所蕴涵

对 $R(U, F)$ 的任一关系 r 中的任意两个元组 t, s :

若 $t[X] = s[X]$, 由于 $X \rightarrow Y$, 有 $t[Y] = s[Y]$;

再由 $Y \rightarrow Z$, 有 $t[Z] = s[Z]$, 所以 $X \rightarrow Z$ 为 F 所蕴涵, 传递律得证。

根据 A1、A2、A3 这三条推理规则, 可以得到下面三条很有用的推理规则。

合并规则(union rule): 由 $X \rightarrow Y, X \rightarrow Z$, 有 $X \rightarrow YZ$ 。

伪传递规则(pseudo transitivity rule): 由 $X \rightarrow Y, WY \rightarrow Z$, 有 $XW \rightarrow Z$ 。

分解规则(decomposition rule): 由 $X \rightarrow Y$ 及 $Z \subseteq Y$, 有 $X \rightarrow Z$ 。

根据合并规则和分解规则, 很容易得到这样一个重要事实。

引理 6.1 $X \rightarrow A_1 A_2 \cdots A_k$ 成立的充分必要条件是 $X \rightarrow A_i$ 成立 ($i = 1, 2, \cdots, k$)。

定义 6.12 在关系模式 $R(U, F)$ 中为 F 所逻辑蕴涵的函数依赖的全体叫作 F 的闭包(closure), 记为 F^+ 。

人们把自反律、传递律和增广律称为 Armstrong 公理系统。Armstrong 公理系统是有效的、完备的。其中, 有效性是指由 F 出发, 根据 Armstrong 公理系统推导出来的每一个函数依赖一定在 F^+ 中; 完备性是指 F^+ 中的每一个函数依赖, 必定可以由 F 出发根据 Armstrong 公理系统推导出来。

要证明完备性, 首先要解决如何判定一个函数依赖是否属于由 F 根据 Armstrong 公理系统推导出来的函数依赖的集合。当然, 如果能求出这个集合, 问题就解决了。但不幸的是, 这是一个 NP 完全问题。例如, 从 $F = \{X \rightarrow A_1, \cdots, X \rightarrow A_n\}$ 出发, 至少可以推导出 2^n 个不同的函数依赖。为此引入了下面的概念。

定义 6.13 设 F 为属性组 U 上的一组函数依赖, X 为 U 的子集, $X \subseteq U, X_F^+ = \{A \mid X \rightarrow A \text{ 能够由 } F \text{ 根据 Armstrong 公理系统导出}\}$, X_F^+ 称为 X 关于函数依赖集 F 的闭包。

由引理 6.1 容易得出引理 6.2。

引理 6.2 设 F 为属性组 U 上的一组函数依赖, $X, Y \subseteq U, X \rightarrow Y$ 能由 F 根据 Armstrong 公理系统导出的充分必要条件是 $Y \subseteq X_F^+$ 。

于是, 判定 $X \rightarrow Y$ 是否能由 F 根据 Armstrong 公理系统导出的问题就转化为求出 X_F^+ , 判定 Y 是否为 X_F^+ 的子集的问题。这个问题由算法 6.1 解决了。

算法 6.1 求属性集 $X (X \subseteq U)$ 关于 U 上的函数依赖集 F 的闭包 X_F^+ 。

输入: X, F

输出: X_F^+

步骤:

① 令 $X^{(0)} = X, i = 0$ 。

② 求 B , 这里 $B = \{A \mid (\exists V)(\exists W)(V \rightarrow W \in F \wedge V \subseteq X^{(i)} \wedge A \in W)\}$ 。

③ $X^{(i+1)} = B \cup X^{(i)}$ 。

④ 判断 $X^{(i+1)} = X^{(i)}$ 。

⑤ 若 $X^{(i+1)}$ 与 $X^{(i)}$ 相等或 $X^{(i)} = U$, 则 $X^{(i)}$ 就是 X_F^+ , 算法终止。

⑥ 若否, 则 $i=i+1$, 返回第②步。

[例 6.11] 已知关系模式 $R(U, F)$, 其中

$U = \{A, B, C, D, E\}, F = \{AB \rightarrow C, B \rightarrow D, C \rightarrow E, EC \rightarrow B, AC \rightarrow B\}$ 。

求 $(AB)_F^+$, 判断 AB 是否为候选码。

解 由算法 6.1, 设 $X^{(0)} = AB$ 。

计算 $X^{(1)}$, 逐一扫扫 F 集合中各个函数依赖, 找左部为 A 、 B 或 AB 的函数依赖。得到两个: $AB \rightarrow C, B \rightarrow D$ 。于是 $X^{(1)} = AB \cup CD = ABCD$ 。

因为 $X^{(0)} \neq X^{(1)}$, 所以再找出左部为 $ABCD$ 子集的那些函数依赖, 又得到 $C \rightarrow E, AC \rightarrow B$ 。于是 $X^{(2)} = X^{(1)} \cup BE = ABCDE$ 。

因为 $X^{(2)}$ 已等于全部属性集合, $(AB)_F^+ = ABCDE$ 。所以 AB 是候选码。

对于算法 6.1, 令 $a_i = |X^{(i)}|$, $\{a_i\}$ 形成一个步长大于 1 的严格递增的序列, 序列的上界是 $|U|$, 因此该算法最多 $|U| - |X|$ 次循环就会终止。

X_F^+ 的概念以及算法 6.1 求 X_F^+ , 可以判断某一属性或属性组是否为码。关系模式 $R(U, F)$ 中, 如果 $X_F^+ = U$, 则 $X \rightarrow U$ 成立, X 为码。

如何求关系模式 $R(U, F)$ 所有的候选码, 读者可参见二维码内容。

定理 6.2 Armstrong 公理系统是有效的、完备的。

Armstrong 公理系统的有效性可由定理 6.1 得到证明。这里给出完备性的证明。

证明完备性的逆否命题, 即若函数依赖 $X \rightarrow Y$ 不能由 F 从 Armstrong 公理系统导出, 那么它必然不为 F 所蕴涵。它的证明分三步。

① 若 $V \rightarrow W$ 成立, 且 $V \subseteq X_F^+$, 则 $W \subseteq X_F^+$ 。

证 因为 $V \subseteq X_F^+$, 所以有 $X \rightarrow V$ 成立, 于是 $X \rightarrow W$ 成立(因为 $X \rightarrow V, V \rightarrow W$), 所以 $W \subseteq X_F^+$ 。

② 构造一张二维表 r , 它由下列两个元组构成, 可以证明 r 必是 $R(U, F)$ 的一个关系, 即 F 中的全部函数依赖在 r 上成立。

X_F^+			$U - X_F^+$		
1	1	...	0	0	...
1	1	...	1	1	...

若 r 不是 $R(U, F)$ 的关系, 则必由于 F 中有某一个函数依赖 $V \rightarrow W$ 在 r 上不成立所致。由 r 的构成可知, V 必定是 X_F^+ 的子集, 而 W 不是 X_F^+ 的子集, 可是由第①步 $W \subseteq X_F^+$, 矛盾。所以 r 必是 $R(U, F)$ 的一个关系。

③ 若 $X \rightarrow Y$ 不能由 F 从 Armstrong 公理系统导出, 则 Y 不是 X_F^+ 的子集, 因此必有 Y 的子集 Y' 满足 $Y' \subseteq U - X_F^+$, 则 $X \rightarrow Y$ 在 r 中不成立, 即 $X \rightarrow Y$ 必不为 $R(U, F)$ 蕴涵。

Armstrong 公理系统的完备性及有效性说明了“导出”与“蕴涵”是两个完全等价的概念。于是 F^+ 也可以说成是由 F 出发, 借助 Armstrong 公理系统导出的函数依赖的集合。



扩展阅读: 求关系模式 $R(U, F)$ 所有候选码

从蕴涵(或导出)的概念出发,又引出了两个函数依赖集等价和最小依赖集的概念。

定义 6.14 如果 $G^+ = F^+$, 就说函数依赖集 F 覆盖 G (F 是 G 的覆盖, 或 G 是 F 的覆盖), 或 F 与 G 等价。

引理 6.3 $F^+ = G^+$ 的充分必要条件是 $F^+ \subseteq G^+$ 和 $G^+ \subseteq F^+$ 。

证 必要性显然, 只证充分性。

① 若 $F \subseteq G^+$, 则 $X_F^+ \subseteq X_{G^+}^+$ 。

② 任取 $X \rightarrow Y \in F^+$, 则有 $Y \subseteq X_F^+ \subseteq X_{G^+}^+$ 。

所以 $X \rightarrow Y \in (G^+)^+ = G^+$ 。即 $F^+ \subseteq G^+$ 。

③ 同理可证 $G^+ \subseteq F^+$, 所以 $F^+ = G^+$ 。

而要判定 $F \subseteq G^+$, 只需逐一对 F 中的函数依赖 $X \rightarrow Y$ 考察 Y 是否属于 X_G^+ , 即可。因此引理 6.3 给出了判断两个函数依赖集等价的可行算法。

定义 6.15 如果函数依赖集 F 满足下列条件, 则称 F 为一个极小函数依赖集, 亦称为最小依赖集或最小覆盖(minimal covering)。

① F 中任一函数依赖的右部仅含有一个属性。

② F 中不存在这样的函数依赖 $X \rightarrow A$, 使得 F 与 $F - \{X \rightarrow A\}$ 等价。

③ F 中不存在这样的函数依赖 $X \rightarrow A$, X 有真子集 Z 使得 $F - \{X \rightarrow A\} \cup \{Z \rightarrow A\}$ 与 F 等价。

定义 6.15③的含义是, 对于 F 中的每个函数依赖, 它的左部要尽可能简。

[例 6.12] 考察 6.1 节例 6.1 中的关系模式 $\text{Student}(U, F)$, 其中:

$U = \{\text{Sno}, \text{School}, \text{Mname}, \text{Cno}, \text{Grade}\}$,

$F = \{\text{Sno} \rightarrow \text{School}, \text{School} \rightarrow \text{Mname}, (\text{Sno}, \text{Cno}) \rightarrow \text{Grade}\}$

设 $F' = \{\text{Sno} \rightarrow \text{School}, \text{Sno} \rightarrow \text{Mname}, \text{School} \rightarrow \text{Mname}, (\text{Sno}, \text{Cno}) \rightarrow \text{Grade}, (\text{Sno}, \text{School}) \rightarrow \text{School}\}$

根据定义 6.15 可以验证 F 是最小覆盖, 而 F' 不是。因为 $F' - \{\text{Sno} \rightarrow \text{Mname}\}$ 与 F' 等价, $F' - \{(\text{Sno}, \text{School}) \rightarrow \text{School}\}$ 与 F' 等价。

定理 6.3 每一个函数依赖集 F 均等价于一个极小函数依赖集 F_m 。此 F_m 称为 F 的最小依赖集。

证 这是一个构造性的证明, 分三步对 F 进行“极小化处理”, 找出 F 的一个最小依赖集。

① 逐一检查 F 中各函数依赖 $FD_i: X \rightarrow Y$, 若 $Y = A_1 A_2 \cdots A_k$, $k \geq 2$, 则用 $\{X \rightarrow A_j \mid j = 1, 2, \dots, k\}$ 来取代 $X \rightarrow Y$ 。

② 逐一检查 F 中各函数依赖 $FD_i: X \rightarrow A$, 令 $G = F - \{X \rightarrow A\}$, 若 $A \in X_G^+$, 则从 F 中去掉此函数依赖(因为 F 与 G 等价的充要条件是 $A \in X_G^+$)。

③ 逐一取出 F 中各函数依赖 $FD_i: X \rightarrow A$, 设 $X = B_1 B_2 \cdots B_m$, $m \geq 2$, 逐一考查 $B_i (i = 1, 2, \dots, m)$, 若 $A \in (X - B_i)_F^+$, 则以 $X - B_i$ 取代 X (因为 F 与 $F - \{X \rightarrow A\} \cup \{Z \rightarrow A\}$ 等价的充要条件

是 $A \in Z_F^+$, 其中 $Z = X - B_i$ 。

最后剩下的 F 就一定是最小依赖集, 并且与原来的 F 等价。因为对 F 的每一次“改造”都保证了改造前后的两个函数依赖集等价。这些证明很显然, 请读者自行补上。

应当指出, F 的最小依赖集 F_m 不一定是唯一的, 它与对各函数依赖 FD_i 及 $X \rightarrow A$ 中 X 各属性的处置顺序有关。

[例 6.13] $F = \{A \rightarrow B, B \rightarrow A, B \rightarrow C, A \rightarrow C, C \rightarrow A\}$

$$F_{m1} = \{A \rightarrow B, B \rightarrow C, C \rightarrow A\}$$

$$F_{m2} = \{A \rightarrow B, B \rightarrow A, A \rightarrow C, C \rightarrow A\}$$

这里给出了 F 的两个最小依赖集 F_{m1} 、 F_{m2} 。

若改造后的 F 与原来的 F 相同, 说明 F 本身就是一个最小依赖集。因此, 定理 6.3 的证明给出的极小化过程, 也可以看成是检验 F 是否为最小依赖集的一个算法。

两个关系模式 $R_1(U, F)$ 、 $R_2(U, G)$, 如果 F 与 G 等价, 那么 R_1 的关系一定是 R_2 的关系; 反过来, R_2 的关系也一定是 R_1 的关系。所以在 $R(U, F)$ 中用与 F 等价的依赖集 G 来取代 F 是允许的。

6.4 保持函数依赖的模式分解

在对函数依赖的基本性质有了初步了解之后, 可以具体来讨论模式的分解了。本节首先讨论保持函数依赖(functional dependency preserving)的模式分解。

定义 6.16 关系模式 $R(U, F)$ 的一个分解是指

$$\rho = \{R_1(U_1, F_1), R_2(U_2, F_2), \dots, R_n(U_n, F_n)\}$$

其中 $U = \bigcup_{i=1}^n U_i$, 并且没有 $U_i \subseteq U_j$, $1 \leq i \neq j \leq n$, F_i 是 F 在 U_i 上的投影, 即

$$F_i = \{X \rightarrow Y \mid X \rightarrow Y \in F^+ \wedge XY \subseteq U_i\}。$$

先来看一个例子, 说明按定义 6.16, 若只要求 $R(U, F)$ 分解后的各关系模式所含属性的“并”等于 U , 这个限定是很不够的。

一个关系分解为多个关系, 相应地原来存储在一张二维表内的数据就要分散存储到多张二维表中。要使这个分解有意义, 起码的要求是后者不能“丢失”前者的信息。

[例 6.14] 已知关系模式 $R(U, F)$, 其中 $U = \{Sno, School, Mname\}$, $F = \{Sno \rightarrow School, School \rightarrow Mname\}$ 。 $R(U, F)$ 的元组语义是学生 Sno 正在一个学院 $School$ 学习, 其院长姓名是 $Mname$; 并且一个学生 (Sno) 只在一个学院学习, 一个学院只有一名院长。 R 的一个关系示例见表 6.7 所示。

表 6.7 R 的一个关系示例

Sno	School	Mname
S1	D1	张五
S2	D1	张五
S3	D2	李四
S4	D3	王一

由于 R 中存在传递函数依赖 $Sno \xrightarrow{T} Mname$, 它会发生更新异常。例如, 如果 S4 毕业, 删除该条记录, 则 D3 学院的院长王一的信息也就丢掉了。反过来, 如果一个学院 D5 刚刚成立, 尚无在校学生, 那么这个学院院长赵某的信息也无法存入。

现在考虑对 R 进行如下分解:

$$\rho_1 = \{R_1(\{Sno, School\}, \{Sno \rightarrow School\}), R_2(\{Sno, Mname\}, \{Sno \rightarrow Mname\})\}$$

这时, 表 R 被分解为两个新的表 R_1 和 R_2 , 每个新表也都有 4 个元组。

但是, 这样的分解仍然存在插入和删除异常, 其原因就在于原来在 R 中存在的函数依赖 $School \rightarrow Mname$, 现在在 R_1 和 R_2 中都不再存在了, 即这个函数依赖被丢掉了。因此一个合理的要求就是, 分解应“保持函数依赖”。

分解后的模式应该保持与原模式的“等价”, 在此, 我们采用保持函数依赖作为模式等价的一种准则。

定义 6.17 $\rho = \{R_1(U_1, F_1), R_2(U_2, F_2), \dots, R_k(U_k, F_k)\}$ 是关系模式 $R(U, F)$ 的一个保持函数依赖的分解, 如果

$$F^+ = \left(\bigcup_{i=1}^k F_i \right)^+$$

6.3 节引理 6.3 给出了判断两个函数依赖集等价的可行算法, 因此引理 6.3 也可用于判别 R 的分解 ρ 是否保持函数依赖。

算法 6.2 (合成法) 转换为 3NF 的保持函数依赖的分解。

- ① 对 $R(U, F)$ 中的函数依赖集 F 进行极小化处理(处理后得到的依赖集仍记为 F)。
- ② 找出所有不在 F 中出现的属性(记为 U_0), 构成一个关系模式 $R_0(U_0, F_0)$ 。把这些属性从 U 中去掉, 剩余的属性仍记为 U 。
- ③ 若有 $X \rightarrow A \in F$, 且 $XA = U$, 则 $\rho = \{R\}$, 算法终止。
- ④ 否则, 对 F 按具有相同左部的原则分组(假定分为 k 组), 每一组函数依赖所涉及的全部属性形成一个属性集 U_i 。若 $U_i \subseteq U_j (i \neq j)$ 就去掉 U_i 。由于经过了步骤②, 故 $U = \bigcup_{i=1}^k U_i$, 于是 $\rho = \{R_1(U_1, F_1), \dots, R_k(U_k, F_k)\} \cup R_0(U_0, F_0)$ 构成 $R(U, F)$ 的一个保持函数依赖的分解, 并且每个 $R_i(U_i, F_i)$ 均属于 3NF。这里 F_i 是 F 在 U_i 上的投影。

由于 F 只是经过了极小化处理和按照相同左部原则进行分组, 因此分解 ρ 保持函数依赖是显然的。

下面证明每一个 $R_i(U_i, F_i)$ 一定属于 3NF。

设 $F_i = \{X \rightarrow A_1, X \rightarrow A_2, \dots, X \rightarrow A_k\}$, $U_i = \{X, A_1, A_2, \dots, A_k\}$

- ① $R_i(U_i, F_i)$ 一定以 X 为码。
- ② 若 $R_i(U_i, F_i)$ 不属于 3NF, 则必存在非主属性 $A_m (1 \leq m \leq k)$ 及属性组合 Y , $A_m \notin Y$, 使得 $X \rightarrow Y$, $Y \rightarrow A_m \in F_i^+$, 而 $Y \rightarrow X \notin F_i^+$ 。

若 $Y \subset X$, 则与 $X \rightarrow A_m$ 属于最小依赖集 F 相矛盾, 因而 $Y \not\subseteq X$ 。不妨设 $Y \cap X = X_1$, $Y - X =$

$\{A_1, \dots, A_p\}$, 令 $G = F - \{X \rightarrow A_m\}$, 显然 $Y \subseteq X_G^+$, 即 $X \rightarrow Y \in G^+$ 。

可以断言 $Y \rightarrow A_m$ 也属于 G^+ 。因为 $Y \rightarrow A_m \in F_i^+$, 所以 $A_m \in Y_F^+$ 。若 $Y \rightarrow A_m$ 不属于 G^+ , 则在求 Y_F^+ 的算法中, 只有使用 $X \rightarrow A_m$ 才能将 A_m 引入。于是按算法 6.1 必有 j , 使得 $X \subseteq Y^{(j)}$, $Y \rightarrow X$ 成立, 与 $Y \rightarrow X \notin F_i^+$ 矛盾。

于是 $X \rightarrow A_m$ 属于 G^+ , 与 F 是最小依赖集相矛盾, 所以 $R_i(U_i, F_i)$ 一定属于 3NF。

至此, 我们已经有一个算法, 可以在保持函数依赖的情况下, 自动化地将一个关系模式分解为一组子关系模式, 而且这些子关系模式都满足 3NF 范式。

* 6.5 无损连接的模式分解

6.5.1 无损连接的模式分解定义

6.4 节讨论了保持函数依赖作为模式分解等价的一个准则。还可以提出一个更直观的等价准则, 就是“无损连接”(lossless join), 即分解后的关系通过自然连接可以“恢复”原始关系。

本节要讨论的问题是:

- ① 什么是无损连接? 如何判断一个分解是否为无损连接的?
- ② 无损连接的模式分解和保持函数依赖的模式分解之间的关系是什么?
- ③ 实现无损连接模式分解的算法是什么? 分解后关系模式所能达到的规范化程度如何?

先定义一个记号: 设 $\rho = \{R_1(U_1, F_1), \dots, R_k(U_k, F_k)\}$ 是 $R(U, F)$ 的一个分解, r 是

$R(U, F)$ 的一个关系。定义 $m_\rho(r) = \bigbowtie_{i=1}^k \pi_{R_i}(r)$, 即 $m_\rho(r)$ 是 r 在 ρ 中各关系模式上投影的连接。

这里 $\pi_{R_i}(r) = \{t.U_i \mid t \in r\}$ 。

注意: 两个关系之间如果没有共同属性, 那么自然连接就退化为笛卡儿积。

引理 6.4 设 $R(U, F)$ 是一个关系模式, $\rho = \{R_1(U_1, F_1), \dots, R_k(U_k, F_k)\}$ 是 R 的一个分解, r 是 R 的一个关系, $r_i = \pi_{R_i}(r)$, 则

- ① $r \subseteq m_\rho(r)$ 。
- ② 若 $s = m_\rho(r)$, 则 $\pi_{R_i}(s) = r_i$ 。
- ③ $m_\rho(m_\rho(r)) = m_\rho(r)$ 。

证

- ① 证明 r 中的任何一个元组属于 $m_\rho(r)$ 。

任取 r 中的一个元组 t , $t \in r$, 设 $t_i = t.U_i (i = 1, 2, \dots, k)$ 。对 k 进行归纳可以证明 $t_1 t_2 \dots$

$t_k \bigbowtie_{i=1}^k \in \pi_{R_i}(r)$, 所以 $t \in m_\rho(r)$, 即 $r \subseteq m_\rho(r)$ 。

② 由①得到 $r \subseteq m_\rho(r)$, 已设 $s = m_\rho(r)$, 所以 $r \subseteq s$, $\pi_{R_i}(r) \subseteq \pi_{R_i}(s)$ 。现只需证明 $\pi_{R_i}(s) \subseteq \pi_{R_i}(r)$, 就有 $\pi_{R_i}(s) = \pi_{R_i}(r) = r_i$ 。

任取 $S_i \in \pi_{R_i}(s)$, 必有 S 中的一个元组 v , 使得 $v.U_i = S_i$ 。根据自然连接的定义 $v = t_1 t_2 \cdots t_k$, 对于其中每一个 t_i , 必存在 r 中的一个元组 t , 使得 $t.U_i = t_i$ 。由前面 $\pi_{R_i}(r)$ 的定义即得 $t_i \in \pi_{R_i}(r)$ 。又因 $v = t_1 t_2 \cdots t_k$, 故 $v.U_i = t_i$ 。又由上面证得 $v.U_i = S_i$, $t_i \in \pi_{R_i}(r)$, 故 $S_i \in \pi_{R_i}(r)$ 。即 $\pi_{R_i}(s) \subseteq \pi_{R_i}(r)$ 。故 $\pi_{R_i}(s) = \pi_{R_i}(r)$ 。

$$\textcircled{3} m_\rho(m_\rho(r)) = \bigwedge_{i=1}^k \pi_{R_i}(m_\rho(r)) = \bigwedge_{i=1}^k \pi_{R_i}(s) = \bigwedge_{i=1}^k \pi_{R_i}(r) = m_\rho(r)$$

定义 6.18 $\rho = \{R_1(U_1, F_1), \dots, R_k(U_k, F_k)\}$ 是 $R(U, F)$ 的一个分解, 若对 $R(U, F)$ 的任何一个关系 r 均有 $r = m_\rho(r)$ 成立, 则称分解 ρ 具有无损连接性, 将 ρ 简称为无损分解。

直接根据定义 6.18 去鉴别一个分解的无损连接性是不可能的, 算法 6.3 给出了一种判别方法。

算法 6.3 判别一个分解的无损连接性。

$\rho = \{R_1(U_1, F_1), \dots, R_k(U_k, F_k)\}$ 是 $R(U, F)$ 的一个分解, $U = \{A_1, \dots, A_n\}$, $F = \{FD_1, FD_2, \dots, FD_p\}$, 不妨设 F 为最小依赖集, 记 FD_i 为 $X_i \rightarrow A_{li}$ 。

① 建立一张 n 列 k 行的表, 每一列对应一个属性, 每一行对应分解中的一个关系模式。若属性 A_j 属于 U_i , 则在 j 列 i 行交叉处填上 a_j , 否则填上 b_{ij} 。

② 对每一个 FD_i 做下列操作: 找到 X_i 所对应的列中具有相同符号的那些行, 考察这些行中 li 列的元素。若其中有 a_{li} , 则全部改为 a_{li} ; 否则全部改为 b_{mli} 。其中 m 是这些行的行号最小值。

应当注意的是, 若某个 b_{mli} 被更改, 那么该表的 li 列中凡是 b_{mli} 的符号 (不管它是否开始找到的那些行) 均应做相应的更改。

如在某次更改之后, 有一行成为 $a_1, a_2, \dots, a_n$, 则算法终止, ρ 具有无损连接性, 否则 ρ 不具有无损连接性。

对 F 中 ρ 个 FD 逐一进行一次这样的处置, 称为对 F 的一次扫描。

③ 比较扫描前后表有无变化, 如有变化则返回第②步, 否则算法终止。

如果发生循环, 那么前次扫描至少应使该表减少一个符号, 表中符号有限, 因此循环必然终止。

定理 6.4 如果算法 6.3 终止时表中有一行为 $a_1, a_2, \dots, a_n$, 则 ρ 为无损连接分解。

证明从略。

[例 6.15] 已知 $R(U, F)$, $U = \{A, B, C, D, E\}$, $F = \{AB \rightarrow C, C \rightarrow D, D \rightarrow E\}$, R 的一个分解为 $R_1(A, B, C)$, $R_2(C, D)$, $R_3(D, E)$ 。

① 首先构造初始表, 如图 6.9(a) 所示。

② 对 $AB \rightarrow C$, 因各元组的第一、二列没有相同的分量, 所以表不改变。由 $C \rightarrow D$ 可以把 b_{14} 改为 a_4 , 再由 $D \rightarrow E$ 可使 b_{15} 、 b_{25} 全改为 a_5 。最后结果为图 6.9(b)。

表中第一行成为 a_1, a_2, a_3, a_4, a_5 , 所以此分解具有无损连接性。

<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
a_1	a_2	a_3	b_{14}	b_{15}
b_{21}	b_{22}	a_3	a_4	b_{25}
b_{31}	b_{32}	b_{33}	a_4	a_5

(a)

<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
a_1	a_2	a_3	a_4	a_5
b_{21}	b_{22}	a_3	a_4	a_5
b_{31}	b_{32}	b_{33}	a_4	a_5

(b)

图 6.9 分解具有无损连接的一个实例

当关系模式 R 分解为两个关系模式 R_1 、 R_2 时有下面的判定准则。

定理 6.5 对于 $R(U, F)$ 的一个分解 $\rho = \{R_1(U_1, F_1), R_2(U_2, F_2)\}$, 如果 $U_1 \cap U_2 \rightarrow U_1 - U_2 \in F^+$ 或 $U_1 \cap U_2 \rightarrow U_2 - U_1 \in F^+$, 则 ρ 具有无损连接性。

定理的证明留给读者完成。

6.5.2 无损连接的模式分解与保持函数依赖的模式分解之间的关系

保持函数依赖的模式分解与无损连接的模式分解这两个概念到底有什么关系呢? 根据定理 6.5, 例 6.14 分解 ρ_1 具有无损连接, 但 ρ_1 不能保持函数依赖。反过来是否也一样呢? 也就是保持函数依赖的分解也不一定能保持无损连接呢?

我们看一个例子。

$R(\{\text{学号, 姓名, 课程号, 课程名称}\}, \{\text{学号} \rightarrow \text{姓名}, \text{课程号} \rightarrow \text{课程名称}\})$, 以及一个可能的分解 $R_1(\{\text{学号, 姓名}\}, \{\text{学号} \rightarrow \text{姓名}\})$ 和 $R_2(\{\text{课程号, 课程名称}\}, \{\text{课程号} \rightarrow \text{课程名称}\})$, 显然, 这个分解是保持函数依赖的。但是, 由于 R_1 和 R_2 之间不存在公共属性, 因此 R_1 与 R_2 的自然连接就退化为笛卡儿积。因此, 这个分解就不满足无损连接性。

从这两个例子可以看出, 保持函数依赖和无损连接是两个不同的概念。也许, 既保持函数依赖又可以无损连接的分解才是最理想的分解。

6.5.3 既无损连接又保持函数依赖的模式分解算法

算法 6.4 转换为 3NF 的无损连接和保持函数依赖的模式分解。

① 设 X 是 $R(U, F)$ 的码。 $R(U, F)$ 已由算法 6.2 分解为 $\rho = \{R_1(U_1, F_1), R_2(U_2, F_2), \dots, R_k(U_k, F_k)\} \cup R_0(U_0, F_0)$, 令 $\tau = \rho \cup \{R^*(X, F_x)\}$ 。

② 有某个 U_i , 若 $X \subseteq U_i$, 则将 $R^*(X, F_x)$ 从 τ 中去掉; 若 $U_i \subseteq X$, 则将 $R(U_i, F_i)$ 从 τ 中

去掉。

③ τ 就是所求的分解。

由于算法 6.4 是在算法 6.2 结果的基础上进行分解的, 因此, 结果必然也是保持函数依赖的。

$R^*(X, F_x)$ 显然属于 3NF, 算法 6.2 确保分解 ρ 是 3NF 的。因此, 只要判定 τ 的无损连接性就行了。

由于 τ 中必有某关系模式 $R(T)$ 的属性集 $T \supseteq X$, X 是 $R(U, F)$ 的码, 任取 $U-T$ 中的属性 B , 必存在某个 i , 使 $B \in T^{(i)}$ (按算法 6.1)。对 i 施行归纳法可以证明, 由算法 6.3, 表中关系模式 $R(T)$ 所在的行一定可成为 $a_1, a_2, \dots, a_n$ 。

τ 的无损连接性得证。

6.5.4 无损连接的模式分解算法

如果只关注无损连接, 那么可以进一步提高规范化程度。

算法 6.5 (分解法) 转换为 BCNF 的无损连接分解。

① 令 $\rho = \{R(U, F)\}$ 。

② 检查 ρ 中各关系模式是否均属于 BCNF。若是, 则算法终止。

③ 设 ρ 中 $R_i(U_i, F_i)$ 不属于 BCNF, 那么必有 $X \rightarrow A \in F_i^+(A \notin X)$, 且 X 非 R_i 的码。因此, XA 是 U_i 的真子集。对 R_i 进行分解: $\sigma = \{S_1, S_2\}$, $U_{S1} = XA$, $U_{S2} = U_i - \{A\}$, 以 σ 代替 $R_i(U_i, F_i)$, 返回第②步。

由于 U 中属性有限, 因而有限次循环后算法 6.5 一定会终止。

这是一个自顶向下的算法。它自然地形成一棵对 $R(U, F)$ 的二叉分解树。应当指出, $R(U, F)$ 的分解树不一定是唯一的, 这与步骤③中具体选定的 $X \rightarrow A$ 有关。

算法 6.5 最初令 $\rho = \{R(U, F)\}$, 显然 ρ 是无损连接分解, 而以后的分解则由下面的引理 6.5 保证了无损连接性。

引理 6.5 若 $\rho = \{R_1(U_1, F_1), \dots, R_k(U_k, F_k)\}$ 是 $R(U, F)$ 的一个无损连接分解, $\sigma = \{S_1, S_2, \dots, S_m\}$ 是 ρ 中 $R_i(U_i, F_i)$ 的一个无损连接分解, 那么

$\rho' = \{R_1, R_2, \dots, R_{i-1}, S_1, \dots, S_m, R_{i+1}, \dots, R_k\}$ 、

$\rho'' = \{R_1, \dots, R_k, R_{k+1}, \dots, R_n\}$ (ρ'' 是 $R(U, F)$ 包含 ρ 的关系模式集合的分解)

均是 $R(U, F)$ 的无损连接分解。

证明的关键是自然连接的结合律。这里仅给出结合律的证明, 其他部分留给读者自行完成。

引理 6.6 $(R_1 \bowtie R_2) \bowtie R_3 = R_1 \bowtie (R_2 \bowtie R_3)$

证

设 r_i 是 $R_i(U_i, F_i)$ 的关系, $i=1, 2, 3$ 。

设 $U_1 \cap U_2 \cap U_3 = V$;

$$U_1 \cap U_2 - V = X;$$

$$U_2 \cap U_3 - V = Y;$$

$$U_1 \cap U_3 - V = Z \text{ (如图 6.10 所示)}.$$

容易证明 t 是 $(R_1 \bowtie R_2) \bowtie R_3$ 中的一个元组的充要条件是:

T_{R_1} 、 T_{R_2} 、 T_{R_3} 是 t 的连串, 这里 $T_{R_i} \in r_i (i=1, 2, 3)$, $T_{R_1}[V] = T_{R_2}[V] = T_{R_3}[V]$, $T_{R_1}[X] = T_{R_2}[X]$, $T_{R_1}[Z] = T_{R_3}[Z]$, $T_{R_2}[Y] = T_{R_3}[Y]$ 。而这也是 t 为 $R_1 \bowtie (R_2 \bowtie R_3)$ 中的元组的充要条件。于是有

$$(R_1 \bowtie R_2) \bowtie R_3 = R_1 \bowtie (R_2 \bowtie R_3)$$

在 6.2.8 小节中已经指出, 一个关系模式中若存在多值依赖(指非平凡的非函数依赖的多值依赖), 则数据的冗余度大且存在插入、修改、删除异常等问题。为此要消除这种多值依赖, 使模式分离达到一个新的高度 4NF。下面讨论达到 4NF 的具有无损连接性的分解。

定理 6.6 关系模式 $R(U, D)$ 中, D 为 R 中函数依赖和多值依赖的集合。则 $X \twoheadrightarrow Y$ 成立的充要条件是 R 的分解 $\rho = \{R_1(X, Y), R_2(X, Z)\}$ 具有无损连接性, 其中 $Z = U - X - Y$ 。

证 先证充分性。

若 ρ 是 R 的一个无损连接分解, 则对 $R(U, F)$ 的任一关系 r , 有

$$r = \pi_{R_1}(r) \bowtie \pi_{R_2}(r)$$

设 $t, s \in r$, 且 $t[X] = s[X]$, 于是 $t[XY], s[XY] \in \pi_{R_1}(r)$, $t[XZ], s[XZ] \in \pi_{R_2}(r)$ 。由于 $t[X] = s[X]$, 所以 $t[XY] \cdot s[XZ]$ 与 $t[XZ] \cdot s[XY]$ 均属于 $\pi_{R_1}(r) \pi_{R_2}(r)$, 也即属于 r 。令 $u = t[XY] \cdot s[XZ]$, $v = t[XZ] \cdot s[XY]$, 就有 $u[X] = v[X] = t[X]$, $u[Y] = t[Y]$, $u[Z] = s[Z]$, $v[Y] = s[Y]$, $v[Z] = t[Z]$, 所以 $X \twoheadrightarrow Y$ 成立。

再证必要性。

若 $X \twoheadrightarrow Y$ 成立, 对于 $R(U, D)$ 的任一关系 r , 任取 $\omega \in \pi_{R_1}(r) \pi_{R_2}(r)$, 则必有 $t, s \in r$, 使得 $\omega = t[XY] \cdot s[XZ]$, 由于 $X \twoheadrightarrow Y$ 对 $R(U, D)$ 成立, ω 应当属于 r , 所以 ρ 是无损连接分解。

定理 6.6 给出了对 $R(U, D)$ 的一个无损的分解方法。若 $R(U, D)$ 中 $X \twoheadrightarrow Y$ 成立, 则 R 的分解 $\rho = \{R_1(X, Y), R_2(X, Z)\}$ 具有无损连接性。

算法 6.6 达到 4NF 的具有无损连接性的分解。

首先使用算法 6.5 得到 R 的一个达到 BCNF 的无损连接分解 ρ , 然后对某一 $R_i(U_i, D_i)$, 若不属于 4NF, 则可按定理 6.6 进行分解, 直到每一个关系模式均属于 4NF 为止。定理 6.6 和引理 6.5 保证了最后得到的分解的无损连接性。

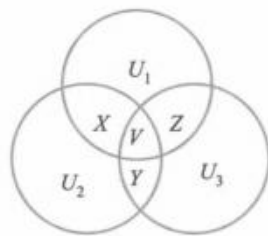


图 6.10 引理 6.6 三个关系属性的示意图

本章小结

本章在函数依赖(FD)、多值依赖(MVD)的范畴内讨论了关系模式的规范化。主要讨论关系规范化理论、函数依赖公理系统以及关系模式的分解算法。

对于一个模式的分解是多种多样的，但是分解后产生的模式应与原模式等价。

人们从不同的角度去观察问题，对“等价”的概念形成了不同的分解准则：

- ① 分解具有无损连接性。
- ② 分解要保持函数依赖。
- ③ 分解既要保持函数依赖，又要具有无损连接性。

按照不同的分解准则，模式所能达到的分离程度各不相同，各种范式就是对分离程度的测度，其基本思想可用图 6.11 来表示。



图 6.11 关系模式规范化小结

算法 6.2(合成法)是保持函数依赖的模式分解算法，分解后的各个关系模式可以达到 3NF，但不一定达到 BCNF。算法 6.4 是既无损连接又保持函数依赖的模式分解算法，分解后的各个关系模式可以达到 3NF。算法 6.5(分解法)是具有无损连接的模式分解，分解后的各个关系模式可以达到 BCNF。如果函数依赖集中有多值依赖，则算法 6.6 是具有无损连接的模式分解，分解后的各个关系模式可以达到 4NF。

2020 年度图灵奖获得者 Jeffrey D. Ullman 曾经表达过这样的观点：每一个进行数据库设计的人员都必须了解什么是函数依赖，也必须知道当进行规范化操作的同时会损失什么；如果不掌握背后的一些理论知识，即使是一些专业人士也无法理解基于数据库开发的应用系统在今后实际运行时可能产生问题的原因。因此他强调人们要更加认真对待理论，**重视理论对数据库设计的指导作用**。

最后还要指出，规范化理论为数据库设计提供了理论的指南和工具，但是在实际应用场景中并不是规范化程度越高模式就越好，必须结合应用环境和现实世界的具体需求合理地设计数据库模式。

习 题 6

1. 理解并给出下列术语的定义：

函数依赖, 部分函数依赖, 完全函数依赖, 传递依赖, 候选码, 超码, 主码, 外码, 全码, 1NF, 2NF, 3NF, BCNF, 多值依赖, 4NF。

2. 建立一个包含系、学生、班级、学会等信息的关系数据库。

描述学生的属性有: 学号、姓名、出生日期、系名、班号、宿舍区;

描述班级的属性有: 班号、专业名、系名、人数、入校年份;

描述系的属性有: 系名、系号、系办公室地点、人数;

描述学会的属性有: 学会名、成立年份、地点、人数。

有关语义如下: 一个系有若干专业, 每个专业每年只招一个班, 每个班有若干学生。一个系的学生住在同一个宿舍区。每个学生可参加若干学会, 每个学会会有若干学生。学生参加某学会会有一个入会年份。

请给出关系模式, 写出每个关系模式的最小依赖集, 指出是否存在传递函数依赖, 对于函数依赖左部是多属性的情况, 讨论函数依赖是完全函数依赖还是部分函数依赖。

指出各关系的候选码、外部码, 并说明是否全码存在。

3. 试由 Armstrong 公理系统推导出下面三条推理规则。

① 合并规则: 若 $X \rightarrow Z$, $X \rightarrow Y$, 则有 $X \rightarrow YZ$ 。

② 伪传递规则: 由 $X \rightarrow Y$, $WY \rightarrow Z$, 有 $XW \rightarrow Z$ 。

③ 分解规则: $X \rightarrow Y$, $Z \subseteq Y$, 有 $X \rightarrow Z$ 。

4. 给定关系模式 $R(U, F)$, 其中 $U = \{A, B, C, D, E\}$, 请回答如下问题:

如果存在函数依赖 $B \rightarrow D$, $DE \rightarrow C$, $EC \rightarrow B$, 列出 R 中所有的码, 并给出主属性、非主属性。

5. 试举出三个多值依赖的实例。

6. 考虑关系模式 $R(U, F)$, $U = \{A, B, C, D, E\}$, 请回答下面的问题:

① 若 A 是 R 的候选码, R 具有函数依赖 $BC \rightarrow DE$, 那么在什么条件下 R 属于 BCNF?

② 如果存在函数依赖 $F = \{A \rightarrow B, BC \rightarrow D, DE \rightarrow A\}$, 列出 R 的所有码。

③ 如果存在函数依赖 $F = \{A \rightarrow B, BC \rightarrow D, DE \rightarrow A\}$, R 属于 3NF 还是 BCNF?

7. 下面的结论哪些是正确的? 哪些是错误的? 对于错误的结论请给出理由或一个反例说明之。

① 任何一个二目关系是属于 3NF 的。

② 任何一个二目关系是属于 BCNF 的。

③ 任何一个二目关系是属于 4NF 的。

④ 当且仅当函数依赖 $A \rightarrow B$ 在 R 上成立, 关系 $R(A, B, C)$ 等于其投影 $R_1(A, B)$ 和 $R_2(A, C)$ 的连接。

⑤ 若 $R.A \rightarrow R.B$, $R.B \rightarrow R.C$, 则 $R.A \rightarrow R.C$ 。

⑥ 若 $R.A \rightarrow R.B$, $R.A \rightarrow R.C$, 则 $R.A \rightarrow R.(B, C)$ 。

⑦ 若 $R.B \rightarrow R.A$, $R.C \rightarrow R.A$, 则 $R.(B, C) \rightarrow R.A$ 。

⑧ 若 $R.(B, C) \rightarrow R.A$, 则 $R.B \rightarrow R.A$, $R.C \rightarrow R.A$ 。

8. 证明:

① 如果 R 是 BCNF 关系模式, 则 R 是 3NF 关系模式, 反之则不然。

② 如果 R 是 3NF 关系模式, 则 R 一定是 2NF 关系模式。

9. 为什么直接根据定义 6.18 去鉴别一个分解的无损连接性是不可能的?

参考文献 6

[1] CODD E F. Normalized data base structure: a brief tutorial[C]. Proceedings. of ACM SIGMOD Workshop on

Data Description, Access and Control, 1971; 1-17.

[2] CODD E F. Further normalized data base relational model[J]. Data Base Systems; Courant Computer Science Symposia Series. Englewood Cliffs; Prentice-Hall, 1972, 6.

[3] ARMSTRONG W W. Dependency structures of data base relationships[C]. Proceedings of International Conference on IFIP Congress, 1974.

Armstrong 在文献[3]中提出了函数依赖公理系统,并给出了正确性和完备性证明。

[4] BERNSTEIN P. Synthesizing third normal form relations from functional dependencies[J]. ACM Transactions on Database Systems, 1976, 1(4): 277-298.

文献[4]给出了关系数据库设计中达到第三范式的综合算法。

[5] ZANILOLO C A. Analysis and design of relational schemata for database systems[D]. Los Angeles; University of California, 1976.

Zaniolo 在他的博士论文中提出了多值依赖的概念。

[6] FAGIN R. Multivalued dependencies and a new normal form for relational database[J]. ACM Transactions on Database Systems, 1977, 2(3): 262-278.

Fagin 在文献[6]中讨论并定义了第四范式。

[7] BEERI C, FAGIN R, HOWARD J H. A complete axiomatization for functional and multivalued dependencies in database relations[C]. Proceedings of the 1977 ACM SIGMOD International Conference on Management of Data, 1977.

文献[7]研究了函数依赖和多值依赖的公理系统及其正确性和完备性。

[8] BERNSTEIN P A, GOODMAN N. What does Boyce-Codd normal form do?[C]. Proceedings of the 6th International Conference on Very Large Data Bases, 1980, 6: 245-259.

[9] AHO A, BEERI C, ULLMAN J D. The theory of joins in relational databases[J]. ACM Transactions on Database Systems, 1979, 4(3): 297-314.

文献[9]分析了无损连接性的性质。

[10] MAIER D. Theory of relational databases[M]. Rockville; Computer Science Press, 1983.

文献[10]中 Maier 对关系依赖理论进行了综合讨论,给出了许多有关函数依赖的定理及证明,介绍了求与给定函数依赖集等价的最小函数依赖集的方法等。

[11] KENT W. Consequences of assuming a universal relation[J]. ACM Transactions on Database Systems, 1981, 6(4): 539-556.

[12] ULLMAN J D. On Kent's "consequences of assuming a universal relation"[J]. ACM Transactions on Database Systems, 1983, 8(4): 637-643.

[13] BOHANNON P, FAN W F, GEERTS, et al. Conditional functional dependencies for data cleaning[C]. 2007 IEEE 23rd International Conference on Data Engineering, 2007: 746-755.

文献[13]中给出了条件函数依赖的定义及相应的推理规则,条件函数依赖是在函数依赖的基础上加入条件约束扩展而来,已应用于数据清洗等问题中。

本章知识点讲解微视频：



函数依赖



多值依赖



范式体系



逻辑蕴涵与推理
系统



模式分解与算法

数据库设计是数据库应用系统设计的重要组成部分，也是应用系统开发的数据基础。其目标是设计数据库的各级模式并建立数据库，主要包括需求分析、概念结构设计、逻辑结构设计、物理结构设计、数据库实施、数据库运行与维护6个阶段。

本章导读

本章详细介绍了关系数据库设计的6个阶段及其使用的技术和方法，重点讨论了需求分析与数据字典、概念结构设计和逻辑结构设计(E-R图到关系模型的转换)。

本章概括了计算机学科中抽象、理论和设计三个学科形态，以“高校本科教务管理”^①信息系统为主线，抽象出“学生学籍管理”“学生选课管理”“教师教学管理”三个应用场景，阐述了如何在理论的指导下，完成分E-R图的设计、集成和优化。通过案例介绍，读者可以从中体会对客观世界从感性认识到理性认识，再由理性认识回到实践中来的科学思维方法。

本章的难点是分析实际应用问题中的用户需求，设计合理的概念结构(E-R图)，优化并构建数据库的逻辑模式和物理模式。因此，在学习的过程中一定要结合真实的案例，了解相应案例的业务背景、应用环境，设计(整理)用户的详细应用需求，并反复练习概念结构的设计。

7.1 数据库设计概述

在数据库领域内，通常把使用数据库的各类信息系统都称为数据库应用系统。例如，以数据库为基础的各种管理信息系统、办公自动化系统、地理信息系统、电子政务系统、电子商务系统等。

数据库设计(database design)，广义地讲，是数据库及其应用系统的设计，即设计整个数

^① “高校本科教务管理”信息系统E-R图及其诸关系模式在本书附录中给出。